week03_data_split_and_technical_EDAl

February 6, 2026

0.1 Exploratory Data Analysis

[27]:
```
# 1) Go to repo
%cd /home/jupyter-guotq/HelpHerInvest

# 2) Sync latest from GitHub before editing
!git pull --rebase origin main
```

```
/home/jupyter-guotq/HelpHerInvest
From https://github.com/tongyuguo/HelpHerInvest
 * branch            main       -> FETCH_HEAD
Already up to date.
```

[28]:
```
import numpy as np
import pandas as pd
import yfinance as yf
import warnings
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path

warnings.filterwarnings("ignore")
```

[29]:
```
import os

# list available data files

repo_dir = "."
!ls -lah {repo_dir}/Data
```

```
total 17M
drwxr-xr-x 2 jupyter-guotq jupyter-guotq  186 Feb  6 01:58 .
drwxr-xr-x 9 jupyter-guotq jupyter-guotq 4.0K Feb  6 02:11 ..
-rw-r--r-- 1 jupyter-guotq jupyter-guotq 2.4M Jan 31 01:15
dependent_variables.csv
-rw-r--r-- 1 jupyter-guotq jupyter-guotq 2.8M Feb  6 01:58 final_dataset.csv
-rw-r--r-- 1 jupyter-guotq jupyter-guotq 1.1M Feb  6 01:58 final_dataset.csv.zip
-rw-r--r-- 1 jupyter-guotq jupyter-guotq 674K Jan 31 01:15
independent_variables.csv
```

```
-rw-r--r-- 1 jupyter-guotq jupyter-guotq 9.2M Jan 29 18:07
stock_symbols_new.csv.zip
-rw-r--r-- 1 jupyter-guotq jupyter-guotq 296K Jan 30 21:22 testing_small.csv.zip
```

[30]:
```python
import zipfile
# Create Data directory path
DATA_DIR = Path(repo_dir) / "Data"
# Load the dataset
dataset_path= DATA_DIR / "final_dataset.csv.zip"
print(dataset_path)
csv_file_name = "final_dataset.csv"


with zipfile.ZipFile(dataset_path, 'r') as zf:
    with zf.open(csv_file_name) as file_handle:
        # Read the file-like object directly into pandas
        merged_df = pd.read_csv(file_handle)
```

```
Data/final_dataset.csv.zip
```

[31]:
```python
# Load the dataset
idx = len(merged_df.index)
print(f"Dataset loaded with {idx} rows.")

train_num = int(idx * 0.7)
val_num = int(idx * 0.15)
test_num = idx - train_num - val_num
print(f"Train set: {train_num} rows"
      f"\nValidation set: {val_num} rows"
      f"\nTest set: {test_num} rows")

training_data = merged_df.iloc[:train_num]
training_data.tail()
```

```
Dataset loaded with 9018 rows.
Train set: 6312 rows
Validation set: 1352 rows
Test set: 1354 rows
```

[31]:
```
            Date Ticker    mom_1m    mom_3m    mom_6m   mom_12m  \
6307  2021-06-30     KO -0.664909 -0.725725 -1.178227 -0.658839
6308  2021-06-30     MS -0.312631  0.928687  1.010281  1.192557
6309  2021-06-30    AZN  0.251339  1.117750  0.222708 -0.836243
6310  2021-06-30   CSCO -0.412172 -0.747593  0.079779 -0.862740
6311  2021-06-30    CAT -1.992531 -1.725354  0.110645  0.704804

      mom_12m_ex_1m  rel_3m_spy  rel_6m_spy  rel_12m_spy    vol_3m    vol_6m  \
6307      -0.543735   -0.725725   -1.178227    -0.658839 -1.460881 -1.031446
```

```
6308       1.301193    0.928687    1.010281     1.192557  0.227042 -0.099618
6309      -0.916816    1.117750    0.222708    -0.836243  1.017971  0.052143
6310      -0.812849   -0.747593    0.079779    -0.862740 -0.814890 -0.779071
6311       1.364474   -1.725354    0.110645     0.704804 -0.271047 -0.169314

      drawdown_6m  drawdown_12m  pct_above_200dma   adj_close  fwd_excess  \
6307     0.215115       0.27088         -0.963440   47.294609   -0.005968
6308     0.514905       0.55748          1.392062   78.470230   -0.031146
6309     0.514905       0.55748          0.068075  110.269676   -0.154453
6310     0.514905       0.55748          0.012837   46.186298    0.095101
6311    -1.572799      -1.43837         -0.509397  199.377823   -0.156697

      fwd_return
6307    0.111061
6308    0.085883
6309   -0.037424
6310    0.212130
6311   -0.039668
```

[32]:
```python
# Perform basic EDA
count_nas = training_data.isna().sum()
print("Missing values per column:")
print(count_nas)

# Summary statistics
training_data.describe()
```

```
Missing values per column:
Date                  0
Ticker                0
mom_1m                0
mom_3m               96
mom_6m              240
mom_12m             526
mom_12m_ex_1m       526
rel_3m_spy           96
rel_6m_spy          240
rel_12m_spy         526
vol_3m               96
vol_6m              241
drawdown_6m         192
drawdown_12m        479
pct_above_200dma    388
adj_close             0
fwd_excess            0
fwd_return            0
dtype: int64
```

[32]:

	mom_1m	mom_3m	mom_6m	mom_12m	mom_12m_ex_1m \
count	6312.000000	6216.000000	6072.000000	5786.000000	5786.000000
mean	0.002863	0.003779	0.005591	0.008231	0.007929
std	1.006955	1.007301	1.008082	1.008781	1.008683
min	-4.941975	-4.616448	-4.855368	-3.909592	-3.897538
25%	-0.568056	-0.579037	-0.569720	-0.529152	-0.536417
50%	-0.047140	-0.071575	-0.105291	-0.147831	-0.152973
75%	0.522415	0.506885	0.454105	0.346506	0.364365
max	6.164418	6.110228	6.533410	6.526852	6.353619

	rel_3m_spy	rel_6m_spy	rel_12m_spy	vol_3m	vol_6m \
count	6216.000000	6072.000000	5786.000000	6216.000000	6071.000000
mean	0.003779	0.005591	0.008231	0.010924	0.010749
std	1.007301	1.008082	1.008781	1.006466	1.006531
min	-4.616448	-4.855368	-3.909592	-1.746743	-1.669030
25%	-0.579037	-0.569720	-0.529152	-0.641027	-0.635937
50%	-0.071575	-0.105291	-0.147831	-0.256803	-0.265028
75%	0.506885	0.454105	0.346506	0.366834	0.377619
max	6.110228	6.533410	6.526852	6.000460	5.679121

	drawdown_6m	drawdown_12m	pct_above_200dma	adj_close	fwd_excess \
count	6120.000000	5833.000000	5924.000000	6312.000000	6312.000000
mean	-0.002114	-0.001277	0.005295	63.114141	0.034056
std	1.005151	1.005172	1.007927	60.450825	0.233581
min	-6.121311	-6.307731	-5.148334	0.210643	-0.742708
25%	-0.356688	-0.361195	-0.568757	22.894295	-0.083384
50%	0.397425	0.369031	-0.063726	45.524954	0.011648
75%	0.630456	0.639367	0.499260	83.734716	0.111277
max	1.660461	1.562580	5.877712	662.283752	4.020147

	fwd_return
count	6312.000000
mean	0.110603
std	0.249181
min	-0.761706
25%	-0.018780
50%	0.084548
75%	0.200053
max	4.103457

We retrieved the summary statistics for the "fwd_excess" variable of the 50 tickers generated. Additionally, we drilled down to each ticker to and sorted them by the average "fwd_eccces" variable to see which tickers had the highest values. Many of the tickers are highly traded symbols now (PLTR, TSLA, NVDA, etc.), but one thing that jumps out is how PLTR doesn't have as many observations as the other tickers because it is a recently listed symbol.

[33]:
```python
# Analyze the target variable 'fwd_excess'
print(training_data['fwd_excess'].describe())

# By ticker - mean
ticker_group = training_data.groupby("Ticker")['fwd_excess'].describe().
 ↪sort_values('mean', ascending=False)
ticker_group

# By ticker - sum
#ticker_group = training_data.groupby("Ticker")['fwd_excess'].sum().
 ↪sort_values(ascending=False)
#ticker_group
```

```
count    6312.000000
mean        0.034056
std         0.233581
min        -0.742708
25%        -0.083384
50%         0.011648
75%         0.111277
max         4.020147
Name: fwd_excess, dtype: float64
```

[33]:

Ticker	count	mean	std	min	25%	50%	75%	\
TSLA	132.0	0.343586	0.800536	-0.511812	-0.111229	0.086031	0.365226	
NFLX	137.0	0.203107	0.501023	-0.698185	-0.106404	0.151981	0.385885	
NVDA	137.0	0.195739	0.367165	-0.436245	-0.029262	0.085357	0.415260	
AMD	137.0	0.147773	0.502850	-0.742708	-0.204725	0.041220	0.410807	
AVGO	137.0	0.105272	0.165957	-0.196993	-0.028021	0.086841	0.197614	
META	109.0	0.101172	0.218840	-0.293504	-0.025067	0.062503	0.184757	
AMZN	137.0	0.095740	0.205740	-0.321286	-0.041970	0.070517	0.245249	
ASML	137.0	0.093080	0.146323	-0.213727	0.021243	0.095200	0.168726	
MU	137.0	0.091248	0.348412	-0.472740	-0.212027	0.052406	0.326857	
AAPL	137.0	0.084553	0.189100	-0.430331	-0.043496	0.103470	0.213669	
LRCX	136.0	0.082489	0.189970	-0.358800	-0.043723	0.073878	0.212969	
UNH	137.0	0.061534	0.104059	-0.199528	-0.002127	0.055271	0.122415	
MA	137.0	0.061468	0.123499	-0.227333	-0.018367	0.060947	0.135700	
HD	137.0	0.056975	0.106610	-0.181832	-0.029390	0.055351	0.133885	
MSFT	137.0	0.055061	0.112606	-0.178858	-0.025678	0.071170	0.129963	
V	137.0	0.043824	0.118697	-0.236447	-0.022832	0.060026	0.127745	
GOOGL	137.0	0.043584	0.119453	-0.275899	-0.032913	0.031590	0.107088	
COST	137.0	0.040447	0.103260	-0.215480	-0.032032	0.049181	0.104214	
LLY	137.0	0.039597	0.155044	-0.281866	-0.070174	0.042033	0.142176	
NVO	136.0	0.034361	0.168535	-0.455729	-0.072839	0.033267	0.157097	
ABBV	101.0	0.019997	0.139895	-0.374199	-0.069675	0.016518	0.087363	
MS	137.0	0.017154	0.189100	-0.365527	-0.098558	-0.008541	0.134341	

JPM	137.0	0.013140	0.130747	-0.310867	-0.071509	0.000684	0.078843
CAT	137.0	0.009842	0.169436	-0.288168	-0.115459	-0.003376	0.116303
AXP	136.0	0.006312	0.121268	-0.299514	-0.085429	0.020831	0.085120
BAC	137.0	0.004568	0.205949	-0.471089	-0.116828	-0.009026	0.112951
ORCL	137.0	-0.001420	0.103014	-0.238922	-0.074479	0.000980	0.068398
GS	136.0	-0.004437	0.172183	-0.277798	-0.122770	-0.030493	0.064762
BABA	81.0	-0.007257	0.252590	-0.593218	-0.170324	-0.013610	0.165658
SAP	136.0	-0.007582	0.123064	-0.404325	-0.071001	0.003200	0.072925
AZN	137.0	-0.012436	0.134459	-0.343772	-0.100816	-0.012155	0.068342
WMT	137.0	-0.012479	0.115432	-0.271324	-0.096540	-0.018307	0.076524
PM	136.0	-0.012948	0.129421	-0.306727	-0.102532	-0.015953	0.088798
CSCO	137.0	-0.013090	0.147138	-0.393400	-0.115434	0.015670	0.094784
JNJ	137.0	-0.013163	0.093064	-0.213838	-0.089048	-0.013488	0.053065
BRK-B	137.0	-0.013393	0.074213	-0.225964	-0.059383	-0.002022	0.035044
TM	136.0	-0.014292	0.108605	-0.228667	-0.087705	-0.010048	0.057065
MRK	136.0	-0.015412	0.123722	-0.331786	-0.073730	-0.021238	0.051595
PG	137.0	-0.017653	0.109243	-0.302303	-0.082535	-0.027336	0.044782
WFC	137.0	-0.018506	0.176817	-0.585536	-0.098657	-0.010153	0.051747
NVS	136.0	-0.022991	0.104854	-0.265649	-0.106262	-0.015506	0.069911
KO	137.0	-0.023711	0.091391	-0.248644	-0.092211	-0.035513	0.037906
CVX	137.0	-0.030075	0.116919	-0.356030	-0.100413	-0.019923	0.049547
GE	137.0	-0.043557	0.230219	-0.607044	-0.125743	-0.045590	0.064691
XOM	137.0	-0.045409	0.148135	-0.403921	-0.122668	-0.050387	0.013505
IBM	136.0	-0.049794	0.104866	-0.250915	-0.123938	-0.043282	0.019547
HSBC	136.0	-0.062829	0.146881	-0.611188	-0.135322	-0.079996	-0.008727
PLTR	9.0	-0.077458	0.445365	-0.575596	-0.322350	-0.092884	-0.033100

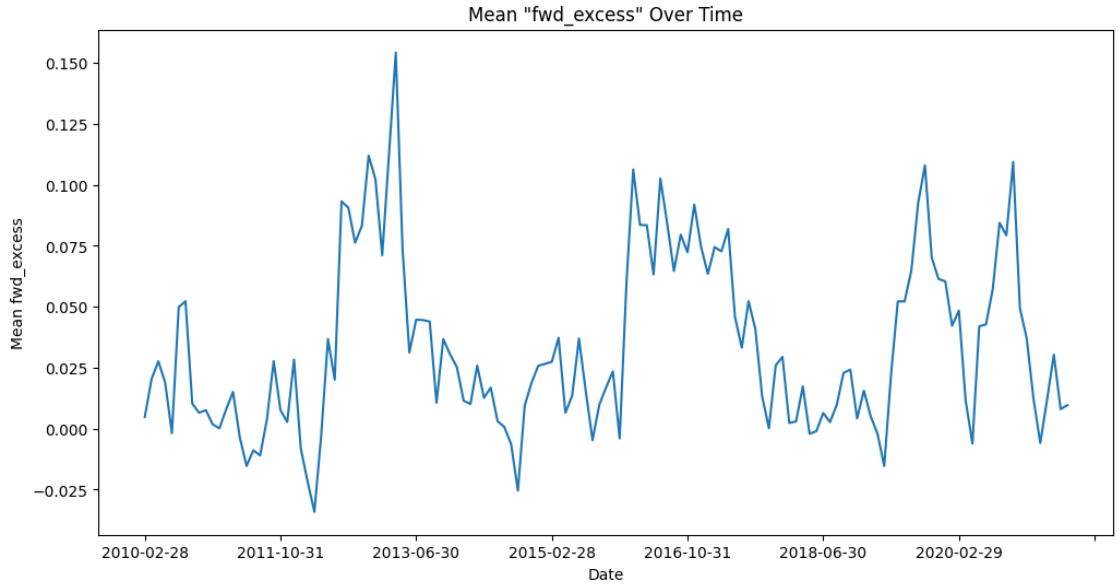
Ticker	max
TSLA	4.020147
NFLX	2.376051
NVDA	1.423459
AMD	2.323218
AVGO	0.566537
META	1.033359
AMZN	0.640745
ASML	0.513613
MU	1.122790
AAPL	0.705514
LRCX	0.610807
UNH	0.378020
MA	0.445558
HD	0.340476
MSFT	0.329686
V	0.306520
GOOGL	0.439198
COST	0.339299

```
LLY     0.439670
NVO     0.389702
ABBV    0.474103
MS      0.577069
JPM     0.415049
CAT     0.436448
AXP     0.404100
BAC     0.600587
ORCL    0.260872
GS      0.572993
BABA    0.612718
SAP     0.294701
AZN     0.453215
WMT     0.267803
PM      0.278477
CSCO    0.304018
JNJ     0.231869
BRK-B   0.163082
TM      0.377056
MRK     0.345451
PG      0.289536
WFC     0.828957
NVS     0.205118
KO      0.191718
CVX     0.305307
GE      0.918638
XOM     0.540011
IBM     0.219798
HSBC    0.332654
PLTR    0.986837
```

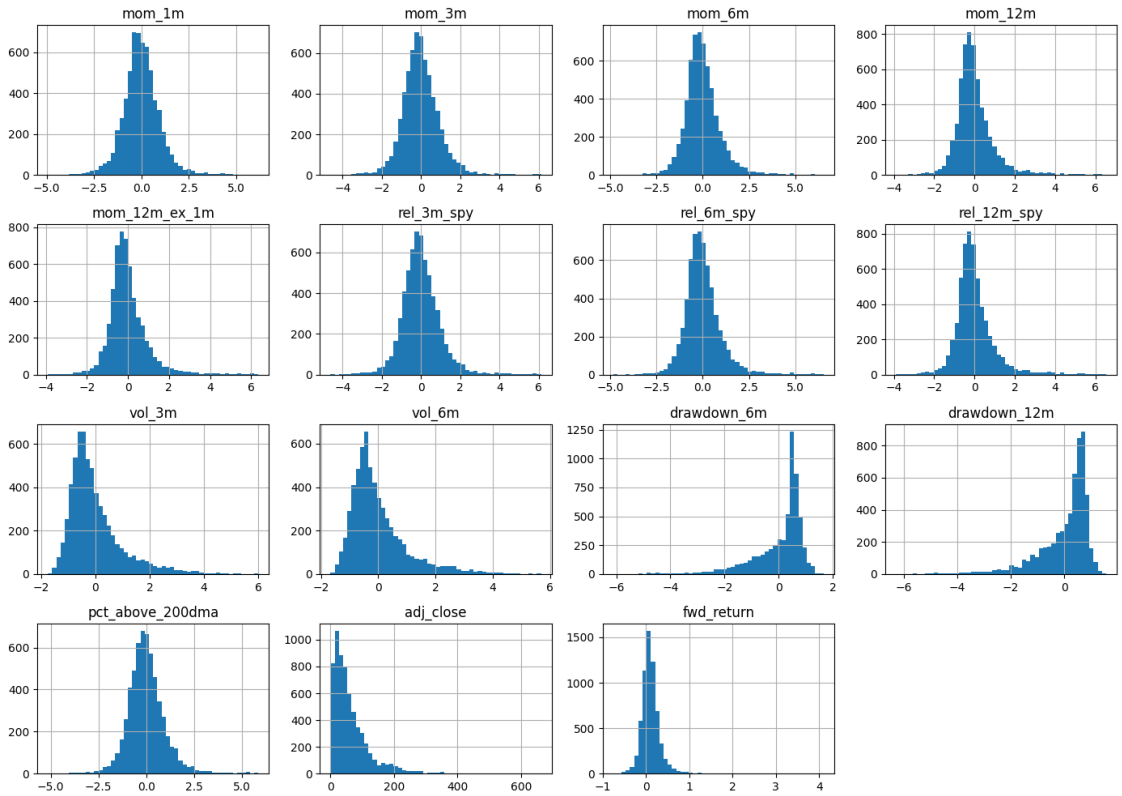
Below is a chart that displays the average "fwd_excess" over the course of the review period. This determines the average across all 50 tickers and provides a good representation of what this portfolio of 50 names could potentially return against the S&P 500 if the weights were evenly distributed. This is something interesting to note.

[34]:
```
# Average 'fwd_excess' over time
date_group = training_data.groupby('Date')['fwd_excess'].mean()
date_group.plot(title='Mean "fwd_excess" Over Time', figsize=(12,6),␣
 ↪ylabel='Mean fwd_excess', xlabel='Date')
```

[34]:
```
<Axes: title={'center': 'Mean "fwd_excess" Over Time'}, xlabel='Date',
ylabel='Mean fwd_excess'>
```



[35]:
```
# Histogram of variables

cols = training_data.columns.drop(['fwd_excess','Date'])
training_data[cols].hist(figsize=(14, 10), bins=50)
plt.tight_layout()
plt.show()
```

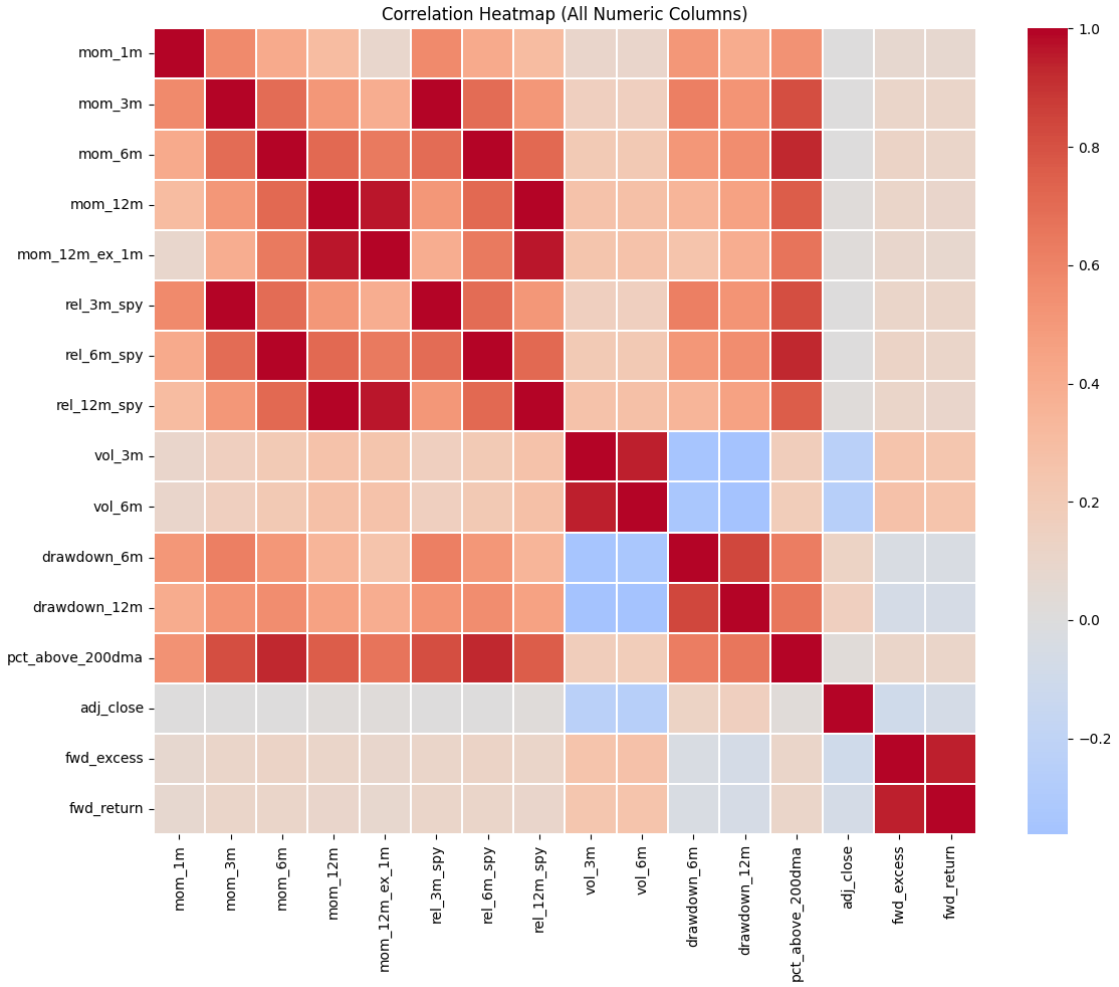


The histograms above display the distribution of values for each variable. Many of these are fairly normally distributed, but the volatility variables are slightly right-skewed and the drawdown variables are slightly left-skewed.

[36]:
```
# Correaltion Anlaysis

# select only numeric columns for correlation
numeric_data = training_data.select_dtypes(include=[np.number])
corr = training_data.select_dtypes(include='number').corr()

fwd_excess_corr = numeric_data.corrwith(numeric_data['fwd_excess']).
 ↪sort_values()
print("Correlation of features with 'fwd_excess':")
print(fwd_excess_corr.tail(15))

plt.figure(figsize=(12, 10))
sns.heatmap(corr, cmap='coolwarm', center=0, linewidths=0.2)
plt.title('Correlation Heatmap (All Numeric Columns)')
plt.tight_layout()
plt.show()
```

```
Correlation of features with 'fwd_excess':
drawdown_12m       -0.061281
```

```
drawdown_6m        -0.036104
mom_1m              0.078982
mom_12m_ex_1m       0.086621
rel_12m_spy         0.105410
mom_12m             0.105410
mom_3m              0.106769
rel_3m_spy          0.106769
pct_above_200dma    0.110647
rel_6m_spy          0.117103
mom_6m              0.117103
vol_3m              0.251590
vol_6m              0.275100
fwd_return          0.948606
fwd_excess          1.000000
dtype: float64
```

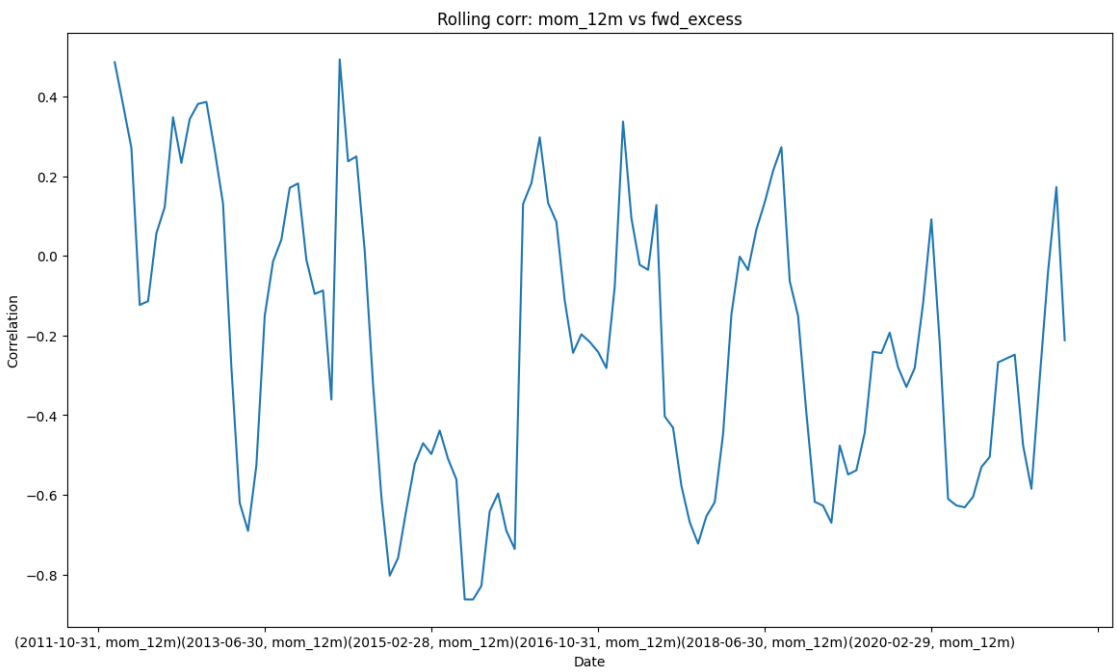


The correlation analysis does not provide strong positive or negative results for the "fwd_excess"

variable. I am curious to see if this is consistent with most tickers and if the timeframe is too wide. Adjusting these parameters may produce stronger correlations.

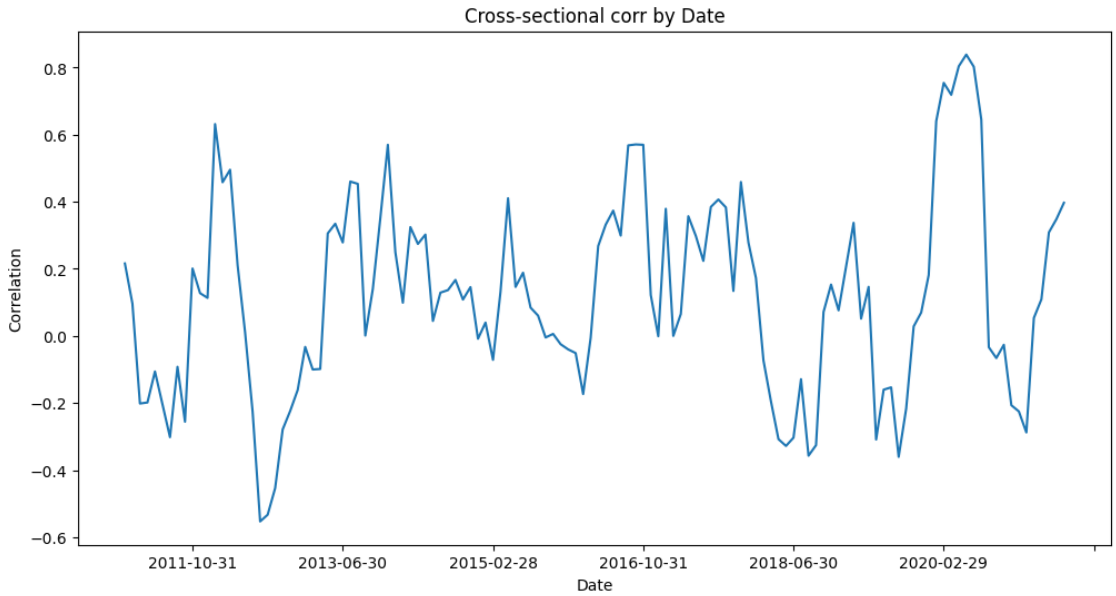
[37]:
```
# Example rolling correlation by date
tmp = training_data.reset_index()
tmp = tmp.sort_values('Date')
roll = tmp.groupby('Date')[['mom_12m','fwd_excess']].mean().rolling(12).corr().
 ↪iloc[0::2,1]
roll.plot(title='Rolling corr: mom_12m vs fwd_excess', figsize=(14,8),␣
 ↪ylabel='Correlation', xlabel='Date')
```

[37]:
```
<Axes: title={'center': 'Rolling corr: mom_12m vs fwd_excess'}, xlabel='Date',
ylabel='Correlation'>
```



[38]:
```
'''
Cross - sectional analysis per date
For each month, examine signal effectiveness cross-sectionally
Rank features by how well they explain fwd_excess
'''

# Cross-sectional correlation per date
cs_corr = tmp.groupby('Date').apply(lambda g: g['mom_12m'].
 ↪corr(g['fwd_excess']))
cs_corr.plot(title='Cross-sectional corr by Date',figsize=(12,6),␣
 ↪ylabel='Correlation', xlabel='Date')
```

[38]: <Axes: title={'center': 'Cross-sectional corr by Date'}, xlabel='Date', ylabel='Correlation'>



[39]:
```
'''
Potential Outlier influence
Extreme fwd_excess tails: which tickers/dates drive them?
Winsorization sensitivity checks
'''

q = training_data['fwd_excess'].quantile([0.01, 0.99])
tail = training_data[(training_data['fwd_excess']<=q.iloc[0]) |␣
↪(training_data['fwd_excess']>=q.iloc[1])]
tail.reset_index().groupby('Ticker').size().sort_values(ascending=False).
↪head(10)
```

[39]:
```
Ticker
AMD     22
TSLA    21
NFLX    20
NVDA    13
GE      11
MU       9
BABA     7
WFC      6
BAC      4
PLTR     3
dtype: int64
```

[40]:
```
# 4) Check what changed
!git status -sb

# 5) Stage + commit
#!git add -A
#!git commit -m "hopefull final testing of working and output directory"

# 6) Push
#!git push
```

```
## main…origin/main
```